AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI

Generative AI Micro Project (7CS1018)

Submitted by

Chalini M – 2023BCSE07AED440

Spandana Basava Kulal – 2023BCSE07AED441

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (DA)

Under the Supervision of

Dr. Marimganti Srinivas



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU NOVEMBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

DEPARTMENT OF CSE (AI/ML)

CERTIFICATE

This is to certify that the In-House Project work entitled **"AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI"** is the Bonafide work done by Chalini M (2023BCSE07AED440), Spandana Basava Kulal (2023BCSE07AED441) submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in **Computer Science and Engineering** during the year 2026.

Supervisor
Associate Professor
Dept. of CSE (Specialization)

ALLIANCE UNIVERSITY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

This is to declare that the report titled **"AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI"** has been made for the partial fulfilment of the Programme - Bachelor of Technology in **Computer Science and Engineering**, under the guidance of **Dr. Marimganti Srinivas**. We confirm that this report truly represents my work undertaken as a part of **our Gen AI Micro Project** work. This work is not a replication of work done previously by any other person. We also confirm that the contents of the report and the views contained

therein have been discussed and deliberated with the Mentor.

NAME	REGESTRATION NO.	SIGNATURE
Chalini M	2023BCSE07AED440	
Spandana Basava Kulal	2023BCSE07AED441	

ALLIANCE UNIVERSITY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people that made it possible, whose constant guidance and encouragement crown all the efforts successfully.

We are very thankful to my supervisor **Dr. Marimganti Srinivas, Associate Professor** for his sustained inspiring guidance and cooperation throughout the process of this project. Their wise counsel and valuable suggestions are invaluable.

We would like to thank **Dr. K Rama Lakshmi,** Head of the Department and **Dr. V Vivek**, Deputy Director, for their encouragement and cooperation at various levels of Project.

We avail this chance to express my deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere, and encouragement.

We express my deep sense of gratitude and because of the teaching and non-teaching staff at our department who stood with me during the project and helped me to make it a successful venture.

We place highest regards to my parents, my friends and well-wishers who helped plenty in making the report of this project.

Chalini M

Spandana Basava Kulal

ABSTRACT

The increasing competition in the job market has made resume optimization and interview preparation essential for job seekers. However, candidates often face difficulties in identifying the skills required for a particular job, aligning their resumes with job descriptions, and preparing for interviews effectively. Traditional approaches to career preparation are time-consuming and generally provide limited personalization.

This project presents an **AI Resume & Interview Assistant: Personalized Career Preparation Using Generative AI**, an intelligent application designed to support candidates throughout the job preparation process. The system uses Generative AI and Natural Language Processing techniques to analyze resumes and compare them with target job descriptions. It employs semantic similarity using Nomic Embeddings to identify the degree of alignment between candidate profiles and job requirements. The application also generates personalized resume improvement suggestions, identifies matching and unclear skills, and provides role-specific technical, project-based, and behavioral interview questions.

Furthermore, the system incorporates an adaptive interview chatbot powered by the Llama 3.2 language model through Ollama. It conducts interactive practice interviews by generating follow-up questions based on the candidate's responses. After the interview, the system produces an AI-generated evaluation report highlighting performance, strengths, areas for improvement, and a personalized learning plan. The application is developed using Python and Streamlit, with PyMuPDF for PDF-based resume text extraction.

The proposed system integrates resume analysis, resume improvement, interview preparation, adaptive interview practice, and performance feedback into a unified platform. It aims to make career preparation more personalized, accessible, and efficient while helping candidates identify skill gaps and improve their interview readiness.

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CV	Curriculum Vitae
JD	Job Description
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
UI	User Interface
UX	User Experience
TF-IDF	Term Frequency-Inverse Document Frequency
FAISS	Facebook AI Similarity Search
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
CPU	Central Processing Unit
RAM	Random Access Memory

CONTENTS

	Page No
Title Page	**1**
Certificate	**2**
Declaration	**3**
Acknowledgement	**4**
Abstract	**5**
List of Symbols & Abbreviations	**6**
Contents	**7**
1. Introduction	**8**
2. Problem Statement, Objectives and Scope	**9-11**
2.1 Problem Statement	
2.2 Objectives	
2.3 Scope of the Project	
3. Literature Review / Existing Systems	**12**
4. Proposed Methodology and System Architecture	**13-15**
4.1 Overall Workflow	
4.2 Main Components	
4.3 Processing and Generation Procedure	
5. Tools, Technologies and Knowledge Sources	**16**
5.1 Knowledge Sources / Dataset	
5.2 Reproducibility Details	
6. Implementation	**17**
6.1 Data Preparation	
6.2 Embedding and Similarity Analysis	
6.3 Resume and Job Description Analysis	
6.4 LLM Integration	
6.5 User Interface	
6.6 Error Handling	
7. Experiments, Results and Discussion	**18-20**
7.1 Evaluation Design	
7.2 Results and Discussion	
8. Limitations, Ethical Considerations and Risk Mitigation	**21-22**
8.1 Limitations	
8.2 Ethical Considerations	
8.3 Risk Mitigation Strategies	
9. Conclusion and Future Enhancements	**23**
9.1 Conclusion	
9.2 Future Enhancements	
References	**24**

1. INTRODUCTION

In today's competitive job market, securing employment requires more than academic qualifications. Candidates must present their skills effectively through well-structured resumes and demonstrate their technical knowledge, communication abilities, and problem-solving skills during interviews. However, many job seekers face challenges in preparing resumes according to specific job requirements and practicing for interviews in a structured and personalized manner.

Traditional resume preparation involves manually reviewing job descriptions, identifying relevant skills, modifying resume content, and preparing possible interview questions. This process can be time-consuming and may not always provide sufficient guidance about skill gaps or areas that require improvement. Similarly, conventional interview preparation methods often rely on generic question banks that do not consider an individual's resume, projects, or target job role.

To address these challenges, the **AI Resume & Interview Assistant** is developed as a Generative AI-based application that provides personalized career preparation support through an integrated platform. The system uses Natural Language Processing (NLP), semantic similarity techniques, and Large Language Models (LLMs) to analyze candidate resumes

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

3.1 Problem Statement

In the current competitive employment environment, candidates often face difficulties in preparing resumes that align with specific job descriptions and identifying the skills required for their desired positions. Manually comparing resumes with job requirements, identifying skill gaps, improving resume content, and preparing for interviews can be time-consuming and challenging.

Existing interview preparation resources frequently provide generic questions that do not consider an individual's technical background, projects, skills, or target job role. Additionally, candidates may lack an effective mechanism to evaluate their interview responses and identify areas that require further improvement.

Therefore, there is a need for an integrated AI-based system that can analyze resumes against job descriptions, identify matching and unclear skills, suggest relevant resume improvements, generate personalized interview questions, conduct adaptive practice interviews, and provide performance feedback to support continuous career preparation.

3.2 Objectives

The primary objective of this project is to develop an AI-powered resume and interview assistant that provides personalized career preparation using Generative AI and Natural Language Processing techniques.

The specific objectives are:

1. **Resume Text Extraction:** To extract relevant information from PDF resumes using PyMuPDF for further analysis.
2. **Resume and Job Description Analysis:** To compare candidate resumes with target job descriptions using semantic similarity techniques and identify their alignment.
3. **Skill Matching and Gap Identification:** To identify matching skills, missing or unclear requirements, and weakly represented skills based on resume evidence.
4. **Resume Improvement:** To generate role-specific suggestions and improve existing resume bullet points while maintaining the accuracy of candidate information.

5. **Personalized Interview Preparation:** To generate technical, project-based, and behavioral interview questions according to the candidate's resume and target job role.
6. **Adaptive Interview Practice:** To develop an interactive chatbot that generates relevant follow-up questions based on the candidate's previous responses.
7. **Interview Performance Evaluation:** To evaluate submitted interview responses and generate feedback highlighting strengths and areas for improvement.
8. **Personalized Learning Plan:** To recommend relevant topics, practice questions, and learning activities based on the interview evaluation.

3.3 Scope of the Project

The scope of the proposed system covers the integration of AI-based resume analysis and personalized interview preparation into a single application.

The major functionalities included within the project scope are:

1. **Resume Upload and Processing:** Supports uploading PDF resumes and extracting selectable text for analysis.
2. **Job Description Input:** Allows candidates to enter their target job role and paste the corresponding job description.
3. **Semantic Similarity Analysis:** Uses Nomic Embeddings to calculate semantic similarity between resume content and job description requirements.
4. **Resume Analysis:** Generates a structured report containing resume summaries, job requirements, matching skills, unclear requirements, and overall assessment.
5. **Resume Enhancement Suggestions:** Provides recommendations for improving resume content, formatting, clarity, and role-specific relevance.
6. **Interview Question Generation:** Generates personalized technical, project-based, and behavioral questions.
7. **Interactive Interview Chatbot:** Conducts practice interviews using the Llama 3.2 language model through Ollama and generates adaptive follow-up questions.
8. **Feedback and Evaluation:** Produces AI-generated interview feedback based on submitted answers, including approximate performance assessments and improvement areas.
9. **Learning Recommendations:** Provides a personalized learning plan to help candidates improve their knowledge and interview performance.

10. **Report Download:** Allows users to download generated resume improvement, interview preparation, and evaluation reports.

The application is developed using Python and Streamlit, with Ollama for local language model processing, Nomic Embeddings for semantic similarity, and PyMuPDF for PDF text extraction.

3. LITERATURE REVIEW / EXISTING SYSTEMS

The following research papers provide a foundation for AI-based resume analysis and personalized interview preparation.

Research Paper	**Contribution to Our Project**
Human and LLM-Based Resume Matching: An Observational Study (NAACL 2025)	Studies LLM-based resume evaluation and highlights challenges such as inaccurate ratings and bias.
EZInterviewer: To Improve Job Interview Performance with Mock Interview Generator (WSDM 2023)	Generates personalized interview questions using candidate resumes, job descriptions, and conversation history.
PolyInterview: An LLM-based Platform for Immersive Mock Interview Practice with Comprehensive Multimodal Assessment (2026)	Combines personalized mock interviews, adaptive follow-up questions, and structured performance feedback.

These studies demonstrate the potential of LLMs for resume matching and interview preparation. Our project combines resume analysis, skill-gap identification, resume improvement, adaptive mock interviews, and personalized feedback in one platform.

4. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

4.1 Overall Workflow

The proposed system follows an integrated workflow consisting of resume analysis, job description matching, skill-gap identification, resume improvement, personalized interview preparation, and performance evaluation.

Initially, the candidate uploads a resume in PDF format and enters the target job role along with the job description. The system extracts the resume text using PyMuPDF and processes the provided information. The extracted resume content and job description are converted into semantic embeddings using the Nomic Embedding model. Cosine similarity is used to estimate semantic relevance between the resume and job description.

The Llama 3.2 language model analyzes the extracted information to identify matching skills, missing requirements, relevant experience, and areas requiring improvement. Based on this analysis, the system generates personalized resume improvement suggestions and interview preparation questions. During the mock interview, the system generates questions based on the candidate's resume and target job role. It also generates follow-up questions based on the candidate's previous answers to simulate an interactive interview experience.

Finally, the system evaluates the interview responses and generates a performance report containing scores, strengths, improvement areas, practice questions, and a personalized learning plan.

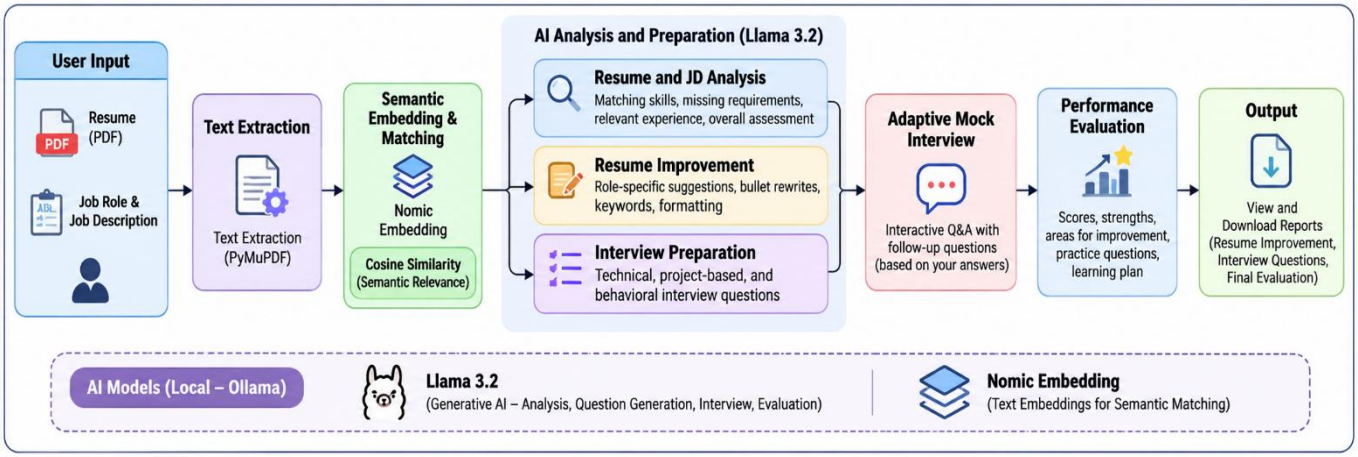


Figure 1: Proposed System Architecture of the AI Resume & Interview Assistant

Figure 1. Proposed System Architecture

4.2 Main Components

Component	Purpose
Resume Upload	Accepts the candidate's resume in PDF format.
Text Extraction	Extracts textual information from the uploaded resume using PyMuPDF.
Job Description Input	Collects the target job role and job description.
Embedding Model	Generates semantic embeddings using Nomic Embedding.
Resume Matcher	Calculates cosine similarity between resume and job description embeddings.
Resume Analyzer	Identifies matching skills, missing requirements, and relevant experience using Llama 3.2.
Resume Improvement Module	Generates suggestions to improve resume content and highlight relevant skills.
Interview Preparation Module	Generates role-specific technical, project-based, and behavioral questions.
Adaptive Interview Module	Generates interview questions and follow-up questions based on candidate responses.
Evaluation Module	Evaluates interview responses and identifies strengths and improvement areas.
User Interface	Displays analysis results, interview interactions, and downloadable reports using Streamlit.

4.3 Processing and Generation Procedure

Step 1: Resume Upload. The candidate uploads a PDF resume through the application interface.

Step 2: Text Extraction. PyMuPDF extracts the textual content from the uploaded resume.

Step 3: Job Description Input. The candidate enters the target job role and corresponding job description.

Step 4: Semantic Matching. The resume and job description are converted into embeddings using Nomic Embedding. Cosine similarity is calculated to estimate their semantic relevance.

Step 5: Resume and Skill Analysis. Llama 3.2 analyzes the resume and job description to identify matching skills, missing requirements, and areas requiring improvement.

Step 6: Resume Improvement. The system generates personalized recommendations, including content improvements and relevant keywords without inventing qualifications or experience.

Step 7: Interview Preparation. The system generates technical, project-based, and behavioral interview questions based on the candidate's profile and target role.

Step 8: Adaptive Mock Interview. The candidate answers generated questions, and the LLM produces relevant follow-up questions based on previous responses.

Step 9: Performance Evaluation. The system analyzes the interview transcript and generates an evaluation report containing performance scores, strengths, weaknesses, and improvement suggestions.

Step 10: Feedback and Output. The candidate receives a downloadable evaluation report and personalized learning recommendations.

5. TOOLS, TECHNOLOGIES AND KNOWLEDGE SOURCES

Layer	Technology Used
Programming Language	Python
Large Language Model	Llama 3.2
AI Model Runtime	Ollama
Embedding Model	Nomic Embedding (nomic-embed-text)
Semantic Similarity	Cosine Similarity
Application Framework	Streamlit
PDF Processing	PyMuPDF
API Communication	Python Requests
Development Environment	Visual Studio Code

5.1 Knowledge Sources / Dataset

The system does not use a separate model-training dataset. It uses user-provided PDF resumes and job descriptions as input sources for analysis. Candidate interview responses are also used to generate personalized feedback and performance evaluation. The system processes this information using locally running AI models through Ollama.

5.2 Reproducibility Details

The application is developed using Python and Streamlit. Llama 3.2 and Nomic Embedding are executed locally through Ollama. PyMuPDF is used for PDF text extraction, while the Requests library communicates with the Ollama API. Reproducing the system requires Python, the required Python packages, Ollama, and the specified models to be installed and configured.

6. IMPLEMENTATION

The AI Resume & Interview Assistant is implemented using Python and Streamlit, integrating locally hosted AI models through Ollama.

6.1 Data Preparation

The system accepts PDF resumes and job descriptions from users. PyMuPDF extracts text from uploaded resumes, which is then prepared for further analysis.

6.2 Embedding and Similarity Analysis

The Nomic Embedding model converts resume content and job descriptions into numerical embeddings. Cosine similarity is used to measure their semantic relevance.

6.3 Resume and Job Description Analysis

Llama 3.2 analyzes resume information and job requirements to identify matching skills, missing or unclear requirements, and relevant experience.

6.4 LLM Integration

Ollama is used to run the Llama 3.2 model locally. The model generates resume improvement suggestions, interview questions, adaptive follow-up questions, and interview evaluation reports.

6.5 User Interface

The application is developed using Streamlit. Its interface provides PDF upload, job description input, resume analysis, improvement suggestions, interview preparation, an interactive interview chatbot, and downloadable evaluation reports.

6.6 Error Handling

The application handles invalid PDF uploads, empty resume content, missing job descriptions, and Ollama API connection or response errors. It also provides error messages when AI-generated operations fail.

7. EXPERIMENTS, RESULTS AND DISCUSSION

7.1 Evaluation Design

The system is evaluated by testing its major functionalities using different resumes and job descriptions. The evaluation focuses on resume analysis, semantic similarity, skill matching, resume improvement, interview question generation, adaptive interview performance, and feedback generation.

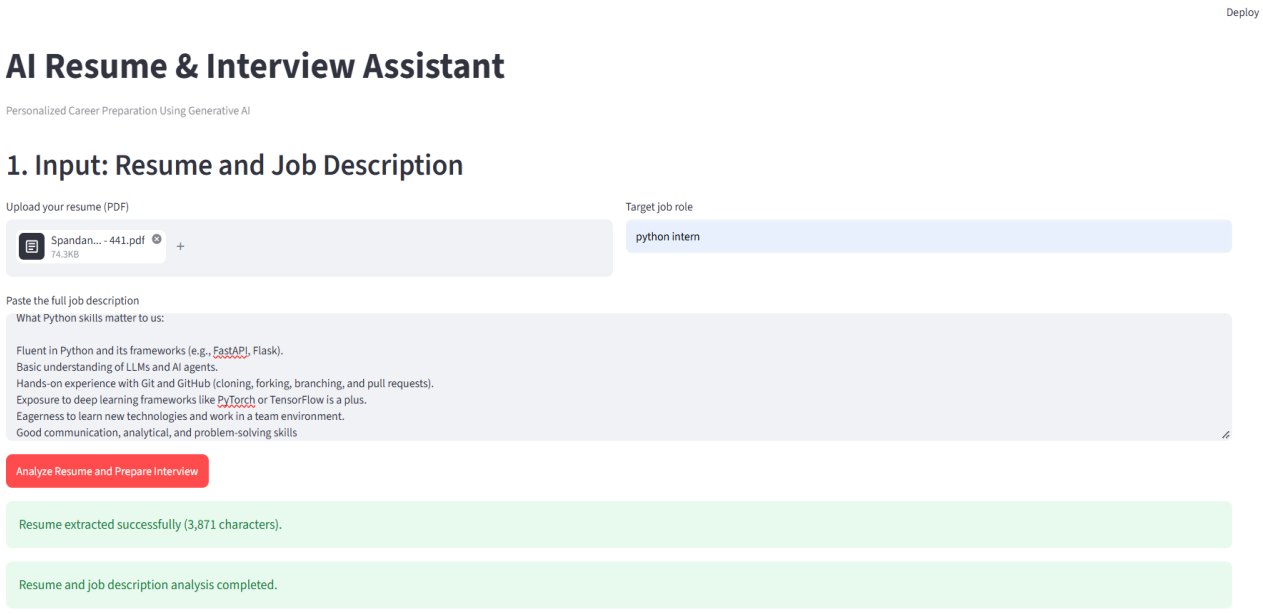
Evaluation Metrics

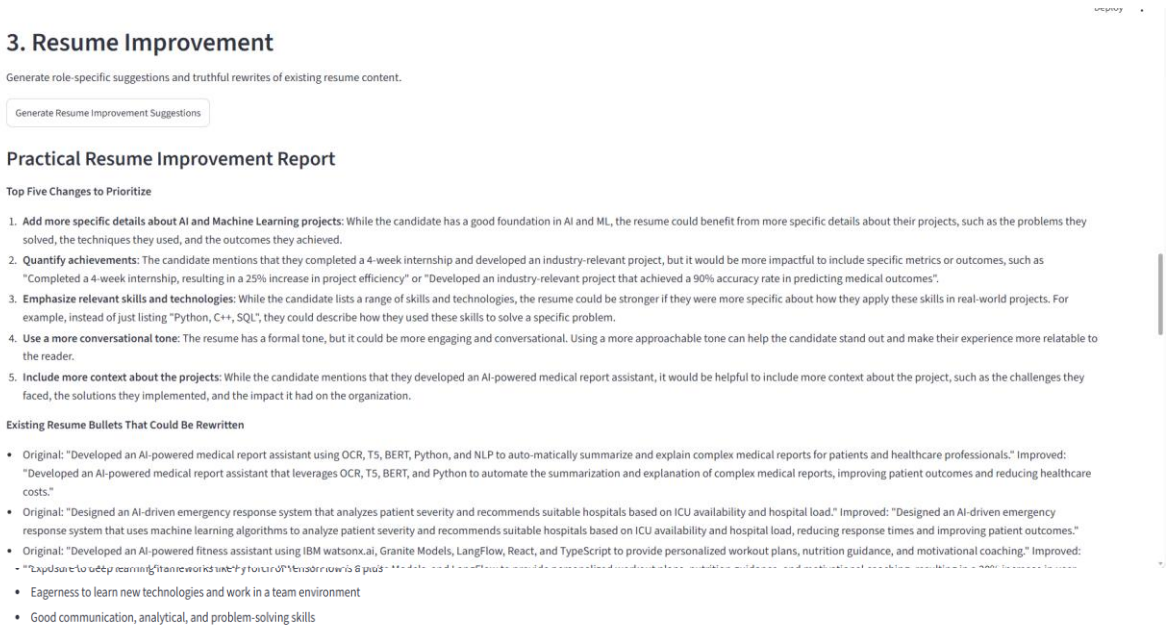
Metric	What It Measures
Semantic Similarity	Measures the relevance between resume content and job description using cosine similarity.
Skill Matching	Checks whether relevant job requirements are identified in the resume.
Resume Improvement Quality	Evaluates the relevance and usefulness of generated resume suggestions.
Question Relevance	Measures how well generated interview questions match the candidate's profile and target role.
Adaptive Interview Performance	Checks whether follow-up questions consider previous candidate responses.
Evaluation Quality	Assesses the relevance and usefulness of generated interview feedback.
Response Time	Measures the time taken by the application to complete AI operations.
User Feedback	Collects feedback on usability and the usefulness of the generated results.

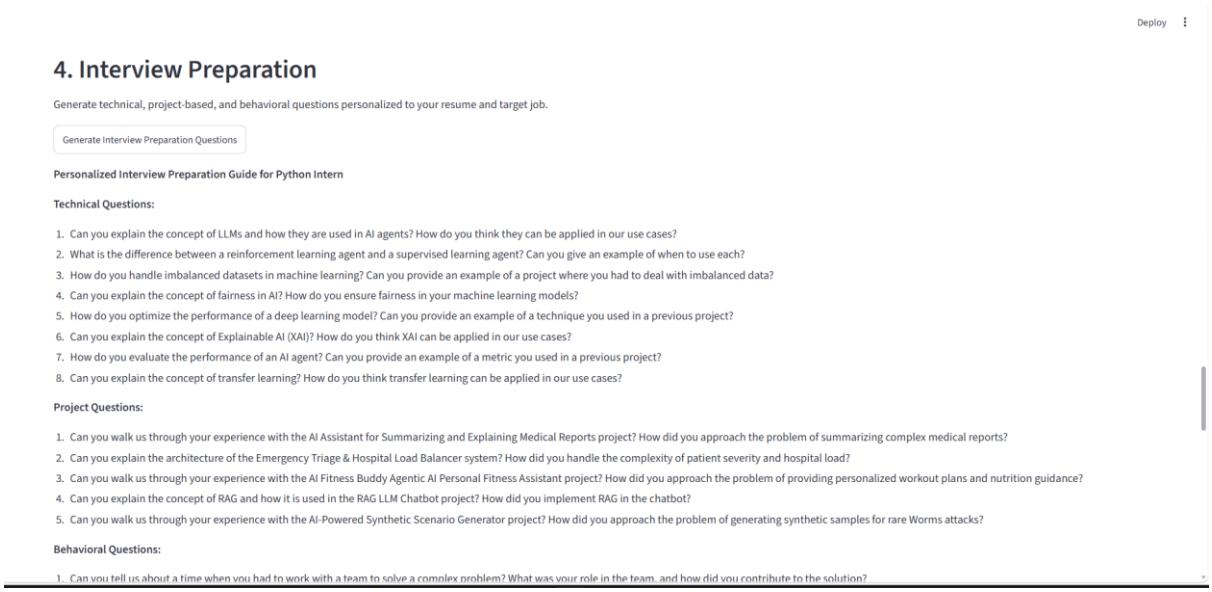
7.2 Results and Discussion

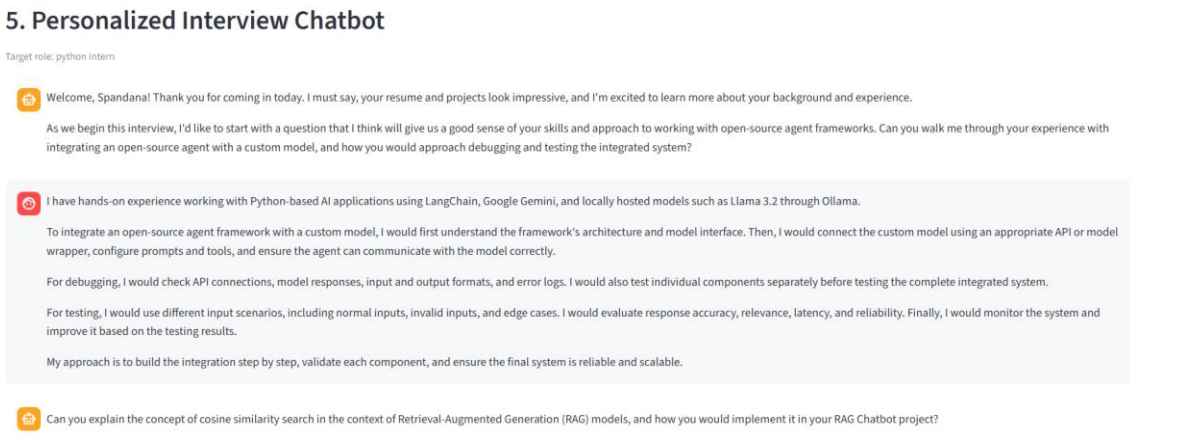
The application successfully demonstrates resume text extraction, resume and job description analysis, personalized resume improvement suggestions, interview question generation, adaptive interview conversations, and AI-generated evaluation reports.

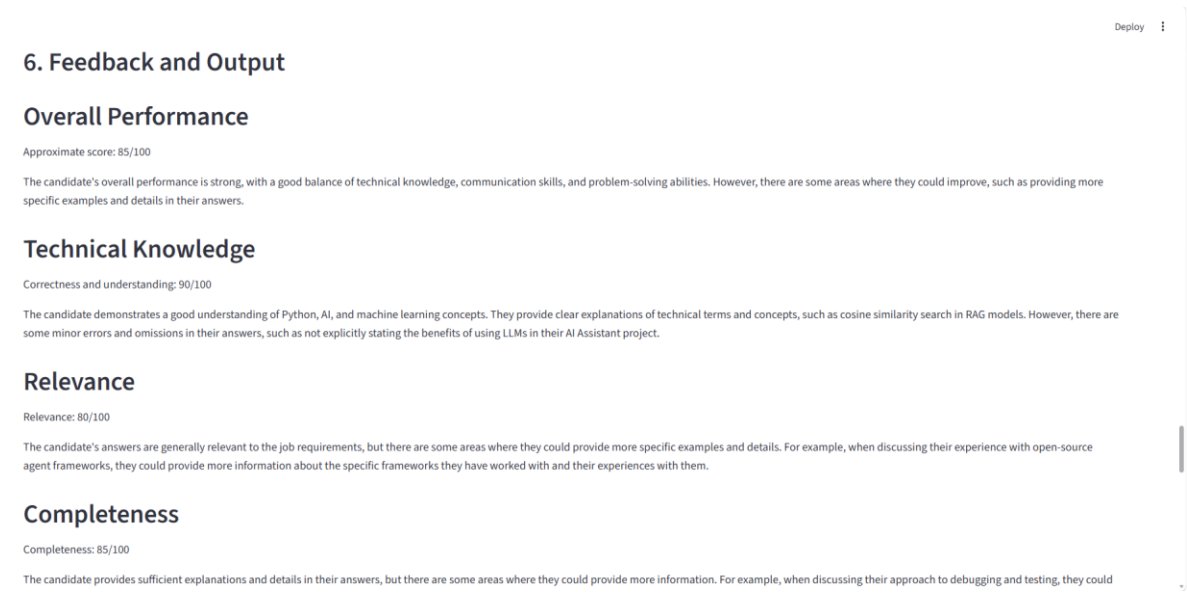
The results demonstrate how Generative AI can support personalized career preparation through an integrated application. However, semantic similarity and AI-generated evaluation scores are approximate and require further validation using a larger test set and user feedback.






Deploy
4. Interview Preparation
Generate technical, project-based, and behavioral questions personalized to your resume and target job.
Generate Interview Preparation Questions
Personalized Interview Preparation Guide for Python Intern
Technical Questions:
1. Can you explain the concept of LLMs and how they are used in AI agents? How do you think they can be applied in our use cases?
2. What is the difference between a reinforcement learning agent and a supervised learning agent? Can you give an example of when to use each?
3. How do you handle imbalanced datasets in machine learning? Can you provide an example of a project where you had to deal with imbalanced data?
4. Can you explain the concept of fairness in AI? How do you ensure fairness in your machine learning models?
5. How do you optimize the performance of a deep learning model? Can you provide an example of a technique you used in a previous project?
6. Can you explain the concept of Explainable AI (XAI)? How do you think XAI can be applied in our use cases?
7. How do you evaluate the performance of an AI agent? Can you provide an example of a metric you used in a previous project?
8. Can you explain the concept of transfer learning? How do you think transfer learning can be applied in our use cases?
Project Questions:
1. Can you walk us through your experience with the AI Assistant for Summarizing and Explaining Medical Reports project? How did you approach the problem of summarizing complex medical reports?
2. Can you explain the architecture of the Emergency Triage & Hospital Load Balancer system? How did you handle the complexity of patient severity and hospital load?
3. Can you walk us through your experience with the AI Fitness Buddy Agentic AI Personal Fitness Assistant project? How did you approach the problem of providing personalized workout plans and nutrition guidance?
4. Can you explain the concept of RAG and how it is used in the RAG LLM Chatbot project? How did you implement RAG in the chatbot?
5. Can you walk us through your experience with the AI-Powered Synthetic Scenario Generator project? How did you approach the problem of generating synthetic samples for rare Worms attacks?
Behavioral Questions:
1. Can you tell us about a time when you had to work with a team to solve a complex problem? What was your role in the team, and how did you contribute to the solution?


5. Personalized Interview Chatbot
Target role: python intern
Welcome, Spandana! Thank you for coming in today. I must say, your resume and projects look impressive, and I'm excited to learn more about your background and experience.
As we begin this interview, I'd like to start with a question that I think will give us a good sense of your skills and approach to working with open-source agent frameworks. Can you walk me through your experience with integrating an open-source agent with a custom model, and how you would approach debugging and testing the integrated system?
I have hands-on experience working with Python-based AI applications using LangChain, Google Gemini, and locally hosted models such as Llama 3.2 through Ollama.
To integrate an open-source agent framework with a custom model, I would first understand the framework's architecture and model interface. Then, I would connect the custom model using an appropriate API or model wrapper, configure prompts and tools, and ensure the agent can communicate with the model correctly.
For debugging, I would check API connections, model responses, input and output formats, and error logs. I would also test individual components separately before testing the complete integrated system.
For testing, I would use different input scenarios, including normal inputs, invalid inputs, and edge cases. I would evaluate response accuracy, relevance, latency, and reliability. Finally, I would monitor the system and improve it based on the testing results.
My approach is to build the integration step by step, validate each component, and ensure the final system is reliable and scalable.
Can you explain the concept of cosine similarity search in the context of Retrieval-Augmented Generation (RAG) models, and how you would implement it in your RAG Chatbot project?


Deploy
6. Feedback and Output
Overall Performance
Approximate score: 85/100
The candidate's overall performance is strong, with a good balance of technical knowledge, communication skills, and problem-solving abilities. However, there are some areas where they could improve, such as providing more specific examples and details in their answers.
Technical Knowledge
Correctness and understanding: 90/100
The candidate demonstrates a good understanding of Python, AI, and machine learning concepts. They provide clear explanations of technical terms and concepts, such as cosine similarity search in RAG models. However, there are some minor errors and omissions in their answers, such as not explicitly stating the benefits of using LLMs in their AI Assistant project.
Relevance
Relevance: 80/100
The candidate's answers are generally relevant to the job requirements, but there are some areas where they could provide more specific examples and details. For example, when discussing their experience with open-source agent frameworks, they could provide more information about the specific frameworks they have worked with and their experiences with them.
Completeness
Completeness: 85/100
The candidate provides sufficient explanations and details in their answers, but there are some areas where they could provide more information. For example, when discussing their approach to debugging and testing, they could

8. Limitations, Ethical Considerations and Risk Mitigation

Although the proposed Retrieval-Augmented Generation (RAG) system provides context-aware responses based on retrieved documents, it has certain limitations that may affect its performance and reliability.

8.1 Limitations

- The system may generate incorrect, incomplete, or irrelevant responses if the retrieved information is insufficient.
- Poor-quality, scanned, or incorrectly formatted PDF documents may affect text extraction and retrieval accuracy.
- The retrieval process may fail to identify relevant information when documents contain complex terminology or ambiguous content.
- The language model may occasionally generate unsupported information (hallucinations) despite being provided with retrieved context.
- The system's performance depends on the quality, completeness, and availability of the uploaded documents.

8.2 Ethical Considerations

The system must be developed and used responsibly to ensure data privacy, transparency, and reliability.

- Only authorized documents should be uploaded and processed.
- Source references should be displayed wherever possible to support answer verification.
- The system should clearly communicate when information is unavailable in the indexed documents.
- Confidential academic, personal, or institutional information must be protected.
- External language model APIs should be used with proper consideration of data privacy and retention policies.
- The system should be tested against prompt injection attacks and malicious instructions embedded in uploaded documents.

8.3 Risk Mitigation Strategies

To minimize potential risks, the following measures are considered:

- Validate uploaded documents before processing.
- Apply appropriate text cleaning and preprocessing techniques.
- Use reliable embedding models and similarity-based retrieval techniques.
- Restrict answer generation to the retrieved context wherever possible.
- Implement error handling for missing documents, API failures, and invalid user inputs.
- Evaluate the system using different test queries to measure retrieval quality and response accuracy.

9. Conclusion and Future Enhancements

9.1 Conclusion

The proposed Retrieval-Augmented Generation (RAG) system demonstrates how Large Language Models (LLMs) can be integrated with document retrieval techniques to provide context-aware responses from a predefined collection of documents.

The system combines document ingestion, text extraction, chunking, embedding generation, vector-based similarity search, and language model-based response generation. By retrieving relevant information before generating responses, the approach aims to improve the relevance and contextual accuracy of answers compared with conventional prompt-based question answering.

The project provides a practical implementation of document-based question answering and demonstrates the potential of Generative AI in academic and administrative information retrieval. However, the reliability of the generated responses depends on document quality, retrieval performance, and the capabilities of the selected language model.

9.2 Future Enhancements

The following improvements can be considered in future versions of the system:

1. **Multilingual Support:** Extend the system to support multiple languages for broader accessibility.
2. **OCR Integration:** Implement Optical Character Recognition (OCR) to extract information from scanned PDF documents and images.
3. **Hybrid Search:** Combine keyword-based retrieval with semantic search to improve document retrieval accuracy.
4. **Conversation Memory:** Maintain conversation history to support contextual follow-up questions.
5. **Access Control:** Introduce role-based authentication to protect confidential documents.
6. **Advanced Evaluation:** Implement automated evaluation metrics to measure retrieval relevance, answer accuracy, and hallucination rates.
7. **Scalability:** Optimize the vector database and deployment architecture to handle larger document collections and multiple users.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[4] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.